LAPORAN PRATIKUM 3

STRUKTUR DATA

“Stack“

disusun Oleh:

Silva Nurifvan

NIM 2511533004

Dosen Pengampu:

DR. WAHYUDI, S.T, M.T

Asisten Pratikum:

Sherly Sukmadira

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, saya dapat menyelesaikan laporan praktikum Struktur Data mengenai Stack ini dengan baik.

Laporan ini disusun sebagai salah satu tugas praktikum untuk memahami konsep dasar struktur data, khususnya struktur data Stack yang menerapkan prinsip LIFO (Last In First Out). Dalam praktikum ini, dilakukan implementasi Stack menggunakan bahasa pemrograman Java sehingga dapat dipahami cara kerja serta penerapannya dalam pengolahan data.

Saya menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan. Oleh karena itu, saya mengharapkan kritik dan saran yang membangun demi perbaikan di masa yang akan datang. Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi pembaca.

Padang, 10 April 2026

Penulis

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan bagian penting dalam dunia pemrograman yang digunakan untuk mengatur, menyimpan, dan mengelola data secara efisien. Salah satu struktur data yang sering digunakan adalah Stack, yaitu struktur data yang bekerja berdasarkan prinsip LIFO (Last In First Out), di mana elemen terakhir yang masuk akan menjadi elemen pertama yang keluar.

Stack banyak digunakan dalam berbagai aplikasi pemrograman, seperti pada fitur undo dan redo, navigasi halaman pada browser, serta dalam proses evaluasi ekspresi matematika dan algoritma rekursif. Oleh karena itu, pemahaman tentang struktur data Stack sangat penting bagi mahasiswa dalam mempelajari dasar-dasar pemrograman dan algoritma.

Dalam implementasinya, Stack memiliki beberapa operasi dasar seperti push (menambahkan data), pop (menghapus data), dan peek (melihat data teratas). Dengan memahami operasi-operasi tersebut, mahasiswa dapat mengelola data secara lebih terstruktur sesuai dengan prinsip kerja Stack. Berdasarkan hal tersebut, praktikum ini dilakukan untuk memahami konsep dasar Stack serta mengimplementasikannya dalam bahasa pemrograman Java, sehingga mahasiswa dapat memahami cara kerja dan penerapannya dalam berbagai permasalahan pemrograman.

1.2 Tujuan Pratikum

1. Memahami konsep dasar struktur data Stack
2. Mengetahui prinsip kerja Stack yaitu LIFO (Last In First Out)
3. Mampu mengimplementasikan Stack dalam bahasa pemrograman Java
4. Memahami operasi dasar Stack seperti push, pop, dan peek
5. Melatih kemampuan dalam mengelola data menggunakan struktur data stack

1.3 Manfaat Pratikum

1. Mahasiswa dapat memahami cara kerja struktur data Stack
2. Meningkatkan kemampuan dalam pemrograman Java
3. Mempermudah penyelesaian masalah yang berkaitan dengan konsep LIFO
4. Menambah pengalaman dalam implementasi struktur data
5. Membantu dalam memahami penerapan Stack pada kasus nyata, seperti undo/redo, navigasi, dan evaluasi ekspresi

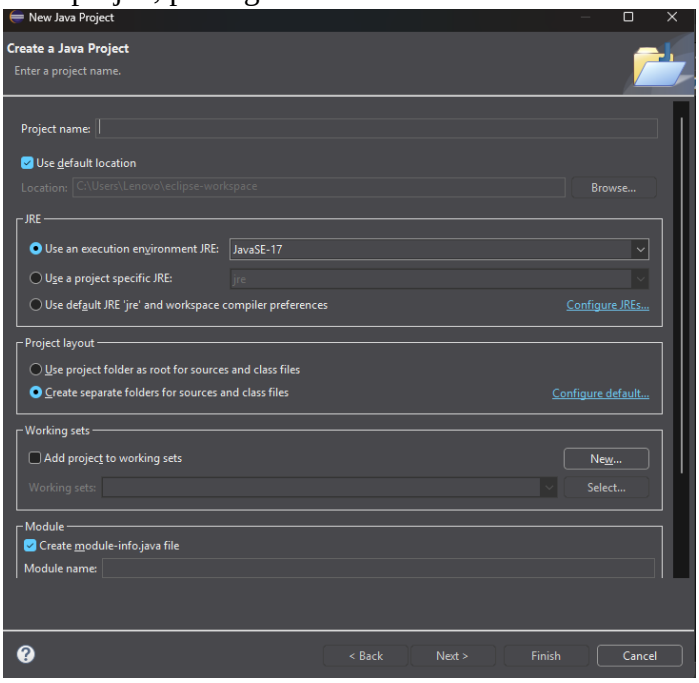
BAB II

PEMBAHASAN

2.1 Pratikum “NilaiMaksimum_2511533004”

Langkah Kerja :

1. Buat projek, package dan class baru



2. Buat kode program berikut

```java
package pekan3_2511533004;

import java.util.Stack;

public class NilaiMaksimum_2511533004 {
        public static int max_2511533004(Stack<Integer> s) {
            Stack<Integer> backup = new Stack<Integer>();
            int maxValue_2511533004 = s.pop();
            backup.push(maxValue_2511533004);

            while (!s.isEmpty()) {
                int next = s.pop();
                backup.push(next);
                maxValue_2511533004 = Math.max(maxValue_2511533004, next);
            }

            while (!backup.isEmpty()) {
                s.push(backup.pop());
            }

            return maxValue_2511533004;
        }

        public static void main(String[] args) {
            Stack<Integer> s = new Stack<Integer>();
            s.push(70);
            s.push(12);
            s.push(20);

            System.out.println("isi stack " + s);
            System.out.println("Stack Teratas " + s.peek());
            System.out.println("Nilai maksimum " + max_2511533004(s));
        }
    }
```

3. Output Program

```
<terminated> NilaiMaksimum_251
isi stack [70, 12, 20]
Stack Teratas 20
Nilai maksimum 70
```

Penjelasan Kode

Import Libarary

Digunakan untuk memakai struktur data Stack di Java

Stack = struktur data dengan konsep LIFO (Last In First Out)

```
import java.util.Stack;
```

Deklarasi class

Mengolah data Stack dan mencari nilai maksimum

```
public class NilaiMaksimum_2511533004 {
```

Method max() (inti program)

Mencari nilai terbesar dalam Stack tanpa menghilangkan isi Stack.

a. Membuat Stack Cadangan
   - Menyimpan sementara data dari stack utama (s)
   - Supaya data tidak hilang saat di-pop
b. Ambil elemen pertama (TOP) pop() → mengambil dan menghapus elemen paling atas
c. Perulangan untuk cek semua elemen Proses:
   - Ambil semua elemen satu per satu (LIFO)
   - Bandingkan untuk cari yang terbesar
   - Simpan ke backup
d. Mengembalikan isi stack
   - Karena sebelumnya semua data sudah di-pop (kosong)
   - Maka dikembalikan lagi ke stack awal
e. Mengembalikan nilai maksimum

```
Stack<Integer> backup = new Stack<Integer>();
int maxValue_2511533004 = s.pop();
backup.push(maxValue_2511533004);

while (!s.isEmpty()) {
    int next = s.pop();
    backup.push(next);
    maxValue_2511533004 = Math.max(maxValue_2511533004, next);
}

while (!backup.isEmpty()) {
    s.push(backup.pop());
}

return maxValue_2511533004;
```

Method main() (Program utama)

a. Membuat Stack Membuat stack kosong
b. Menambahkan data (Push) push() → menambahkan ke atas stack
c. Menampilkan isi stack
d. Menampilkan elemen teratas peek():Melihat elemen paling atas tanpa menghapus

e. Menampilkan nilai maksimum Memanggil method max()

```java
        public static void main(String[] args) {
            Stack<Integer> s = new Stack<Integer>();
            s.push(70);
            s.push(12);
            s.push(20);

            System.out.println("isi stack " + s);
            System.out.println("Stack Teratas " + s.peek());
            System.out.println("Nilai maksimum " + max_2511533004(s));
        }
    }
```

2.2 Pratikum "SiswaStack_2511533004"

Langkah Kerja :

1. Buat Class baru yaitu "SiswaStack_2511533004"
2. Buat kode program berikut

```java
package pekan3_2511533004;

import java.util.ArrayList;

public class Siswastack_2511533004 {
    String nama;
    int nim;

    public Siswastack_2511533004(String nama, int nim) {
        this.nama = nama;
        this.nim = nim;
    }

    @Override
    public String toString() {
        return "Nim: " + nim + ", Nama: " + nama;
    }
}

class SiswaStack {
    private ArrayList<Siswastack_2511533004> stack;

    public SiswaStack() {
        stack = new ArrayList<>();
    }

    public void push(Siswastack_2511533004 mhs) {
        stack.add(mhs);
    }

    public Siswastack_2511533004 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswastack_2511533004 peek() {
```

```java
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }

    public static void main(String[] args) {

        SiswaStack studentStack = new SiswaStack();

        Siswastack_2511533004 mhs1 = new Siswastack_2511533004("Ali", 1);
        Siswastack_2511533004 mhs2 = new Siswastack_2511533004("Boby", 2);
        Siswastack_2511533004 mhs3 = new Siswastack_2511533004("Charles", 3);

        studentStack.push(mhs1);
        studentStack.push(mhs2);
        studentStack.push(mhs3);

        System.out.println("Daftar Siswa di Stack:");
        studentStack.tampilkanSiswa();

        System.out.println("\nData yang di-pop: " + studentStack.pop());

        System.out.println("Data paling atas sekarang: " + studentStack.peek());
    }
}
```

Penjelasan Kode :

Class Siswa I

ni adalah objek/data yang disimpan dalam stack

nama → nama mahasiswa

nim → nomor mahasiswa

a. Konstruktor Untuk mengisi data saat objek dibuat
b. Method toString() Supaya saat di-print hasilnya rapi:

```java
String nama;
int nim;

public Siswastack_2511533004(String nama, int nim) {
    this.nama = nama;
    this.nim = nim;
}

@Override
public String toString() {
    return "Nim: " + nim + ", Nama: " + nama;
}
```

Class SiswaStack_2511533004

a. Atribut

Digunakan sebagai wadah penyimpanan stack, Walaupun pakai ArrayList, kita atur supaya berperilaku seperti stack

b. Konstruktur

Inisialisasi stack kosong

```
class SiswaStack {
    private ArrayList<Siswastack_2511533004> stack;
```

```
public SiswaStack() {
    stack = new ArrayList<>();
}
```

Operasi Sack

a. push (Menambah data) Menambahkan ke atas (TOP)
b. Pop (Menghapus data) Menghapus elemen paling atas
c. Peek (Melihat TOP) Melihat elemen atas tanpa menghapus
d. isEmpty() Untuk cek apakah stack kosong
e. Menampilkan Stack Ini penting, ditampilkan dari atas ke bawah, sesuai konsep stack

```
    public void push(Siswastack_2511533004 mhs) {
        stack.add(mhs);
    }

    public Siswastack_2511533004 pop() {
        if (!isEmpty()) {
            return stack.remove(stack.size() - 1);
        }
        return null;
    }

    public Siswastack_2511533004 peek() {
        if (!isEmpty()) {
            return stack.get(stack.size() - 1);
        }
        return null;
    }

    public boolean isEmpty() {
        return stack.isEmpty();
    }

    public void tampilkanSiswa() {
        for (int i = stack.size() - 1; i >= 0; i--) {
            System.out.println(stack.get(i));
        }
    }
```

Method main()(Driver program)

a. Membuat Satck
b. Membuat data siswa
c. Push ke stack
d. Tampilkan semua
e. Peek
f. Pop
g. Tampilkan lagi

```java
public static void main(String[] args) {

    SiswaStack studentStack = new SiswaStack();

    Siswastack_2511533004 mhs1 = new Siswastack_2511533004("Ali", 1);
    Siswastack_2511533004 mhs2 = new Siswastack_2511533004("Boby", 2);
    Siswastack_2511533004 mhs3 = new Siswastack_2511533004("Charles", 3);

    studentStack.push(mhs1);
    studentStack.push(mhs2);
    studentStack.push(mhs3);

    System.out.println("Daftar Siswa di Stack:");
    studentStack.tampilkanSiswa();

    System.out.println("\nData yang di-pop: " + studentStack.pop());

    System.out.println("Data paling atas sekarang: " + studentStack.peek());
}
```

2.3 Pratikum "stackArray_2511533004"

Langkah kerja :

1. Buat class baru "StackArray_2511533004"
2. Buat kode program berikut

```java
package pekan3_2511533004;

public class stackArray_2511533004 {
    static final int MAX = 1000;
    int top;
    int a[] = new int[MAX];

    boolean isEmpty_2511533004() {
        return (top < 0);
    }
    stackArray_2511533004() {
        top = -1;
    }
    boolean push_2511533004(int x) {
        if (top >= (MAX - 1)) {
            System.out.println("Stack Overflow");
            return false;
        } else {
            a[++top] = x;
            System.out.println(x + " dimasukkan dalam stack");
            return true;
        }
    }
    int pop_2511533004()
    {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
else {
    int x = a[top--];
    return x;
}
    }
int peek_2511533004() {
    if (top < 0) {
        System.out.println("Stack Underflow");
        return 0;
```

```
    }
    else {
        int x = a[top];
        return x;
    }
}

void print_2511533004() {
    for(int i = top;i>-1;i--){
        System.out.print(" "+ a[i]);
    }
}
```

Penjelasan Kode :

Deklarasi Variabel

- MAX → kapasitas maksimal stack (1000 data)
- top→ penunjuk posisi paling atas (TOP)
- a [] → array untuk menyimpan data stack

```
static final int MAX = 1000;
int top;
int a[] = new int[MAX];
```

Method isEmpty()

fungsi, mengecek apakah stack kosong

```
boolean isEmpty_2511533004() {
    return (top < 0);
```

Konstruktor

Inisialisasi stack kosong

```
stackArray_2511533004() {
    top = -1;
}
```

Method push(int X)

Menambahkan data ke dalam stack (ke atas), proses nya:

Cek apakah stack penuh, ika iya → Stack Overflow. artinya, top naik dulu simpan data ke array.

```java
boolean push_2511533004(int x) {
    if (top >= (MAX - 1)) {
        System.out.println("Stack Overflow");
        return false;
    } else {
        a[++top] = x;
        System.out.println(x + " dimasukkan dalam stack");
        return true;
    }
}
```

Method pop()

Menghapus elemen paling atas

- kalau kosong : Stack Underflow
- kalau tidak kosong : ambilkan data di TOP, turunkan top

```java
    int pop_2511533004()
    {
        if (top < 0) {
            System.out.println("Stack Underflow");
            return 0;
        }
else {
    int x = a[top--];
    return x;
```

Method peek()

Melihat elemen paling atas tanpa menghapus

```java
int peek_2511533004() {
    if (top < 0) {
        System.out.println("Stack Underflow");
        return 0;
    }
    else {
        int x = a[top];
        return x;
    }
}
```

Method print()

Menampilkan isi stack dari atas ke bawah

```java
void print_2511533004() {
    for(int i = top;i>-1;i--){
        System.out.print(" "+ a[i]);
    }
}
}
```

2.4 Pratikum “stackArrayDriver_2511533004”

Langkah kerja :

1. Buat class baru “stackArrayDriver_2511533004”
2. Buat kode program berikut

```java
package pekan3_2511533004;

public class stackArrayDriver_2511533004 {

    public static void main(String[] args) {
        stackArray_2511533004 s = new stackArray_2511533004();

        s.push_2511533004(10);
        s.push_2511533004(20);
        s.push_2511533004(30);

        System.out.println(s.pop_2511533004() + " dikeluarkan dari stack");
        System.out.println("Elemen teratas adalah :" + s.peek_2511533004());
        System.out.print("Element pada stack :");
        s.print_2511533004();
    }
}
```

3. Output

```
<terminated> stackArrayDriver_2511533004
10 dimasukkan dalam stack
20 dimasukkan dalam stack
30 dimasukkan dalam stack
30 dikeluarkan dari stack
Elemen teratas adalah :20
Element pada stack : 20 10
```

Penjelasan Kode :

Method main()

```java
public static void main(String[] args) {
```

Membuat objek Stack

Membuat stack kosong

```java
stackArray_2511533004 s = new stackArray_2511533004();
```

Operasi push (menambah data)

Push = menambah ke atas (TOP)

```java
s.push_2511533004(10);
s.push_2511533004(20);
s.push_2511533004(30);
```

Operasi pop (menghapus data)

pop = ambil elemen paling atas

```
System.out.println(s.pop_2511533004() + " dikeluarkan dari stack");
```

Operasi peek (Melihat TOP)

Peek = melihat elemen atas tanpa menghapus

```
System.out.println("Elemen teratas adalah :" + s.peek_2511533004());
```

Menampilkan isi stack

Menampilkan dari atas ke bawah

```
System.out.print("Element pada stack :");
s.print_2511533004();
```

2.5 Pratikum "StackPostfix_2511533004"

Langkah kerja :

1. Buat class baru "StackPostfix_2511533004"
2. Buat kode program berikut

```
package pekan3_2511533004;

import java.util.Scanner;
import java.util.Stack;

public class Stackpostfix_2511533004 {
    public class StackPostfix_2511533004 {
        public static int postfixEvaluate_2511533004(String expression) {
            Stack<Integer> s = new Stack<Integer>();
            Scanner input = new Scanner(expression);
            while (input.hasNext()) {
                if (input.hasNextInt()) { // an operand (integer)
                    s.push(input.nextInt());
                } else { // an operator
                    String operator = input.next();
                    int operand2 = s.pop();
                    int operand1 = s.pop();
                    if (operator.equals("+")) {
                        s.push(operand1 + operand2);
                    } else if (operator.equals("-")) {
                        s.push(operand1 - operand2);
                    } else if (operator.equals("*")) {
                        s.push(operand1 * operand2);
                    } else {
                        s.push(operand1 / operand2);
                    }
                }
            }
            input.close();
            return s.pop();
        }
        public static void main(String[] args) {
            System.out.println("hasil postfix= " + postfixEvaluate_2511533004("5 2 4 * + 7 -"));
        }
    }
}
```

Penjelasan Kode :

Import library

Digunakan untuk:

- Stack → menyimpan angka
- Scanner → membaca string expression

```
import java.util.Scanner;
import java.util.Stack;
```

Method postfixEvaluate()

Menghitung hasil dari ekspresi postfix, cara kerja (konsep stack) :

- Membuat stack kosong
- scanner untuk membaca expression satu per satu

```
Stack<Integer> s = new Stack<Integer>();
Scanner input = new Scanner(expression);
while (input.hasNext()) {
```

Jika angka (operand)

Push ke stack

```
if (input.hasNextInt()) { // an operand (integer)
    s.push(input.nextInt());
```

Jika operator

Pop dua elemen terakhir

```
} else { // an operator
    String operator = input.next();
    int operand2 = s.pop();
    int operand1 = s.pop();
```

Proses operasi

Setelah dihitung, hasil dimasukkan lagi ke stack (push)

```
        } else if (operator.equals("-")) {
            s.push(operand1 - operand2);
        } else if (operator.equals("*")) {
            s.push(operand1 * operand2);
        } else {
            s.push(operand1 / operand2);
        }
    }
}
input.close();
```

Hasil akhir

```
return s.pop();
```

Method main()

Menjalankan program dengan input postfix

```
    public static void main(String[] args) {
        System.out.println("hasil postfix= " + postfixEvaluate_2511533004("5 2 4 * + 7 -"));
    }
}
}
```

BAB III

KESIMPULAN

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa struktur data Stack merupakan struktur data yang bekerja dengan prinsip LIFO (Last In First Out), yaitu elemen yang terakhir dimasukkan akan menjadi elemen pertama yang dikeluarkan. Pada program yang telah dibuat, Stack diimplementasikan dalam beberapa bentuk, yaitu menggunakan array, ArrayList, dan Stack bawaan Java. Setiap implementasi menunjukkan operasi dasar Stack seperti push, pop, peek, dan isEmpty yang digunakan untuk mengelola data secara terstruktur.

Selain itu, Stack juga diterapkan dalam kasus nyata seperti perhitungan ekspresi postfix, di mana Stack digunakan untuk menyimpan operand sementara dan melakukan operasi berdasarkan urutan tertentu. Hal ini menunjukkan bahwa Stack tidak hanya digunakan untuk penyimpanan data, tetapi juga sangat berguna dalam penyelesaian masalah algoritma. Dengan demikian, pemahaman mengenai Stack sangat penting karena dapat membantu dalam pengolahan data secara efisien serta menjadi dasar dalam mempelajari struktur data dan algoritma yang lebih kompleks.

LAPORAN TUGAS

PRAKTIKUM

SOAL :

Pada pekan ini, mahasiswa diminta untuk mengimplementasikan konsep Stack (Tumpukan) yang bekerja dengan prinsip LIFO (Last In, First Out). Tema tugas kali ini adalah Simulasi Browser History. Mahasiswa diwajibkan membuat program yang mensimulasikan perpindahan halaman web. Halaman yang terakhir dibuka akan berada di posisi teratas tumpukan, dan menjadi halaman pertama yang dihapus saat tombol "Back" ditekan.

1. **Class Website (Website_2511533004)**

```java
package pekan2_2511533004;

public class Website_2511533004 {

    private String judul_3004;
    private String url_3004;

    // Constructor
    public Website_2511533004(String judul_3004, String url_3004) {
        this.judul_3004 = judul_3004;
        this.url_3004 = url_3004;
    }

    // Getter
    public String getJudul_3004() {
        return judul_3004;
    }

    public String getUrl_3004() {
        return url_3004;
    }

    // Setter
    public void setJudul_3004(String judul_3004) {
        this.judul_3004 = judul_3004;
    }

    public void setUrl_3004(String url_3004) {
        this.url_3004 = url_3004;
    }
}
```

Gambar 1

Kelas Website_251153004merupakan implementasi dari Abstract Data Type (ADT) yang digunakan untuk merepresentasikan satu data website dalam sistem browser history. Di dalam kelas ini dideklarasikan dua atribut utama yaitu judul_3004 dan url_3004 yang bertipe String, di mana atribut tersebut menyimpan informasi nama halaman website dan alamat URL-nya. Kedua atribut ini menggunakan modifier private, yang berarti tidak dapat diakses secara langsung dari luar kelas, sehingga menerapkan prinsip enkapsulasi untuk menjaga keamanan dan kontrol terhadap data.

Untuk menginisialisasi nilai dari atribut tersebut, kelas ini menyediakan sebuah constructor bernama Website_2511533004 yang menerima dua parameter yaitu judul dan URL. Constructor ini akan langsung mengisi nilai atribut saat objek dibuat, sehingga setiap objek website yang diinstansiasi sudah memiliki data yang lengkap sejak awal tanpa perlu diisi secara terpisah. Selanjutnya, kelas ini menyediakan method getter (selektor) yaitu getJudul_3004 () dan getUrl_3004 () yang berfungsi untuk mengambil nilai dari atribut judul dan URL secara aman. Selain itu, terdapat juga method setter (mutator) yaitu

setJudul_3004 () dan setUrl_3004 () yang digunakan untuk mengubah nilai atribut jika diperlukan setelah objek dibuat. Dengan adanya getter dan setter, akses terhadap atribut menjadi lebih terkontrol dan fleksibel tanpa melanggar prinsip enkapsulasi. Secara keseluruhan, kelas Website_2511533004 berfungsi sebagai model data yang terstruktur untuk menyimpan informasi website, di mana setiap objek yang dibuat dari kelas ini akan mewakili satu halaman web dan dapat digunakan oleh kelas driver untuk dikelola dalam struktur data Stack sebagai bagian dari simulasi browser history.

2. Class Browser (Browser_2511533004)

```java
package pekan2_2511533004;

import java.util.Scanner;
import java.util.Stack;

public class Browser_2511533004 {
    public static void main(String[] args) {
        Scanner input_3004 = new Scanner(System.in);
        Stack<Website_2511533004> history_3004 = new Stack<>();

        int pilihan_3004;

        do {
            System.out.println("\n=== Browser History NIM: 2511533004 ===");
            System.out.println("1. Kunjungi Website (Push)");
            System.out.println("2. Tombol Back (Pop)");
            System.out.println("3. Lihat Halaman Aktif (Peek)");
            System.out.println("4. Cek Status History");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");

            pilihan_3004 = input_3004.nextInt();
            input_3004.nextLine();

            switch (pilihan_3004) {

                case 1:
                    System.out.print("Masukkan Judul: ");
                    String judul_3004 = input_3004.nextLine();

                    System.out.print("Masukkan URL: ");
                    String url_3004 = input_3004.nextLine();

                    Website_2511533004 web_3004 =
                            new Website_2511533004(judul_3004, url_3004);

                    history_3004.push(web_3004);

                    System.out.println("Berhasil mengunjungi halaman!");
                    break;

                case 2:
                    if (!history_3004.isEmpty()) {
                        Website_2511533004 removed_3004 = history_3004.pop();
                        System.out.println("Kembali dari: " + removed_3004.getJudul_3004());
                    } else {
                        System.out.println("History kosong, tidak bisa kembali");
                    }
                    break;

                case 3:
                    if (!history_3004.isEmpty()) {
                        Website_2511533004 current_3004 = history_3004.peek();
                        System.out.println("Halaman aktif:");
                        System.out.println("Judul : " + current_3004.getJudul_3004());
                        System.out.println("URL   : " + current_3004.getUrl_3004());
                    } else {
                        System.out.println("Tidak ada halaman yang dibuka.");
                    }
                    break;

                case 4:
                    System.out.println("Jumlah history: " + history_3004.size());
                    if (history_3004.isEmpty()) {
                        System.out.println("History kosong.");
                    } else {
                        System.out.println("History tidak kosong.");
                    }
                    break;

                case 5:
                    System.out.println("Program selesai.");
                    break;

                default:
                    System.out.println("Pilihan tidak valid!");
```

```
                    System.out.println("Pilihan tidak valid!");
                }

            } while (pilihan_3004 != 5);

            input_3004.close();
        }
    }
```

Kelas Browser_2511533004 merupakan kelas driver yang berfungsi sebagai pusat kendali program untuk mensimulasikan browser history menggunakan struktur data Stack dengan prinsip LIFO (Last In, First Out). Eksekusi program dimulai dari method main, yang menjadi titik awal jalannya program. Di dalamnya dibuat objek Scanner untuk menerima input dari pengguna, serta sebuah Stack bertipe Website_2511533004 yang digunakan untuk menyimpan riwayat halaman website yang dikunjungi.

Program kemudian menggunakan variabel pilihan untuk menyimpan input menu dari pengguna dan membungkus seluruh proses dalam struktur perulangan do-while, sehingga menu akan terus ditampilkan sampai pengguna memilih keluar. Menu yang ditampilkan berisi beberapa fitur utama, dan setiap pilihan pengguna akan diproses menggunakan struktur percabangan switch-case untuk menentukan aksi yang sesuai.

Pada saat pengguna memilih menu kunjungi website (push), program akan meminta input berupa judul dan URL, kemudian membuat objek baru dari kelas Website_2511533004 dan menambahkannya ke dalam Stack menggunakan method .push(). Data yang baru dimasukkan akan berada di posisi paling atas Stack, sesuai dengan konsep LIFO.

Ketika pengguna memilih tombol back (pop), program akan mengecek terlebih dahulu apakah Stack kosong menggunakan .isEmpty() untuk menghindari error. Jika tidak kosong, elemen teratas Stack akan diambil dan dihapus menggunakan .pop(), kemudian ditampilkan judul halaman yang telah 16 ditutup. Hal ini menunjukkan bahwa halaman terakhir yang dibuka adalah yang pertama kali dikeluarkan.

Pada fitur lihat halaman aktif (peek), program kembali mengecek apakah Stack kosong. Jika tidak, maka elemen paling atas Stack akan diambil menggunakan .peek() tanpa menghapusnya, lalu ditampilkan informasi judul dan URL-nya. Fitur ini menggambarkan halaman yang sedang aktif dibuka pada browser.

Untuk fitur cek status history, program menampilkan jumlah data yang tersimpan dalam Stack menggunakan method .size(), serta mengecek apakah Stack kosong dengan .isEmpty(), sehingga pengguna dapat mengetahui kondisi riwayat browsing saat ini.

Terakhir, jika pengguna memilih keluar, perulangan akan dihentikan dan program selesai dijalankan. Secara keseluruhan, kelas ini menggabungkan penggunaan input/output, perulangan, percabangan, dan struktur data Stack, sehingga seluruh fitur seperti push, pop, peek, dan pengecekan status dapat berjalan dengan baik sesuai konsep dasar Stack dalam simulasi browser history.

OUTPUT :

```
Browser_2511533004 [Java Application] C:\Users\Lenovo\.p2\
=== Browser History NIM: 2511533004 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 1
Masukkan Judul: SSO UNAND
Masukkan URL: https://sso.unand.ac.id/login
Berhasil mengunjungi halaman!

=== Browser History NIM: 2511533004 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 2
Kembali dari: SSO UNAND

=== Browser History NIM: 2511533004 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan:
```

```
<terminated> Browser_2511533004 [Java Application] C:
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 3
Tidak ada halaman yang dibuka.

=== Browser History NIM: 2511533004 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 4
Jumlah history: 0
History kosong.

=== Browser History NIM: 2511533004 ===
1. Kunjungi Website (Push)
2. Tombol Back (Pop)
3. Lihat Halaman Aktif (Peek)
4. Cek Status History
5. Keluar
Pilihan: 5
Program selesai.
```